■ Basic Machine Simulator – User Documentation

■ Overview

The **Basic Machine Simulator** is a JavaFX-based graphical simulator that emulates a simplified computer architecture. It allows users to load machine code, observe register and memory states, and control program execution cycle-by-cycle.

■ Console Layout and Components

The simulator interface is divided into five key sections that represent core components of a computer system.

1■■ Register Display (Top-Left Panel)
• **GPR (R0–R3):** General-purpose registers used for arithmetic and logic operations.
• **IXR (X1–X3):** Index registers for indirect addressing and looping operations.
• **PC:** Program Counter — holds address of the next instruction.
• **MAR:** Memory Address Register — holds the address to read/write.
• **MBR:** Memory Buffer Register — temporarily stores data transferred to/from memory.
• **IR:** Instruction Register — contains the current instruction being executed.
• **CC:** Condition Code — displays flags (e.g., zero, negative, overflow).
• **MFR:** Machine Fault Register — indicates hardware or memory-related faults.

2■■ Memory / Cache Display (Top-Right Panel)
Displays memory contents as address-value pairs (in octal). The view automatically updates as instructions execute or when memory values are modified through Load/Store operations.

3■■ Printer (Middle-Right Panel)
Logs system messages, I/O output, and instruction execution events such as [LOAD], [STORE], [RUN], and [HALT].

4■■ Console Input (Bottom-Right)
Used for interactive I/O during program execution (e.g., responding to an IN instruction). Input a value or message and press **Send** to transmit it to the running program.

5■■ Control Panel (Bottom-Left)
Contains main operation buttons for controlling the simulator:

Button	Description
Load	Writes the value in the octal field to memory at the address in MAR.
Store	Reads memory content at MAR into the octal field.
Load+	Same as Load, but increments MAR automatically after each write.
Store+	Same as Store, but increments MAR automatically after each read.
Run	Executes the program continuously until a HALT instruction.
Step	Executes one instruction cycle (fetch-decode-execute).
IPL	Initial Program Load – loads the assembled program from load.ld.
Halt	Manually stops execution.
Console Send	Sends console input to the simulated I/O device.

■ Operating the Simulator

1■■ **Initial Setup:** Ensure a valid **load.ld** file exists (generated by the assembler). Launch the simulator.

2■■ **Loading a Program:** Click **IPL** to load the machine code into memory. The start address and contents will appear in the cache view.

3■■ **Executing the Program:** Press **Run** for continuous execution or **Step** to run one instruction at a time.

4■■ **Manual Operations:** Enter specific addresses or data into MAR/MBR and use **Load** or **Store** to inspect or modify memory.

5■■ **Stopping Execution:** Use **Halt** to stop execution at any time. You may inspect registers or reload a new program using **IPL**.

■ Summary of Interaction Flow

Phase	Action	Tool
Assemble	Run Assembler to generate load.ld	Command Line
Load	Click IPL to load program into memory	GUI
Execute	Press Run or Step to begin execution	GUI
Observe	Watch results in Printer & Registers	GUI
Debug	Inspect and modify registers/memory	GUI